

Freight Rate Prediction

Model and validation report

Spotter AI ML assessment • Anindya Kundu • October 2026

1 Summary

The task is to predict `posted_rate` for 12,000 loads in November–December 2025 from 48,000 labelled loads in January–October 2025. That makes it a forecast one to two months ahead, and the validation is built around that.

- **Model.** A linear model on log price (distance, weight, equipment, `market_index`, smooth geography, a linear time trend and an equipment-specific quarter-end ramp), followed by LightGBM on its residuals. The residual model has no trend features, so it cannot flatten the forward projection.
- **Results.** Fold-average MAE of \$85.2 on raw labels, against \$113.7 for an XGBoost/LightGBM blend (an earlier candidate solution). \$88.5 on the two-months-ahead block (train Jan–Aug, test Sep–Oct). 1.33% MAPE on labels with the corrupted rows removed.
- **Key data issue.** 677 training labels (1.4%) are off by a multiplier (about $\times 3.3$ or $\times 0.29$). They are detected and dropped from training. The validation labels presumably contain the same corruption, which sets a floor on any model's raw error.
- **December chart.** \$839 on 1 December rising to \$881 on the 31st, with a weekly cycle (Thursday peaks) on top of the quarter-end ramp.

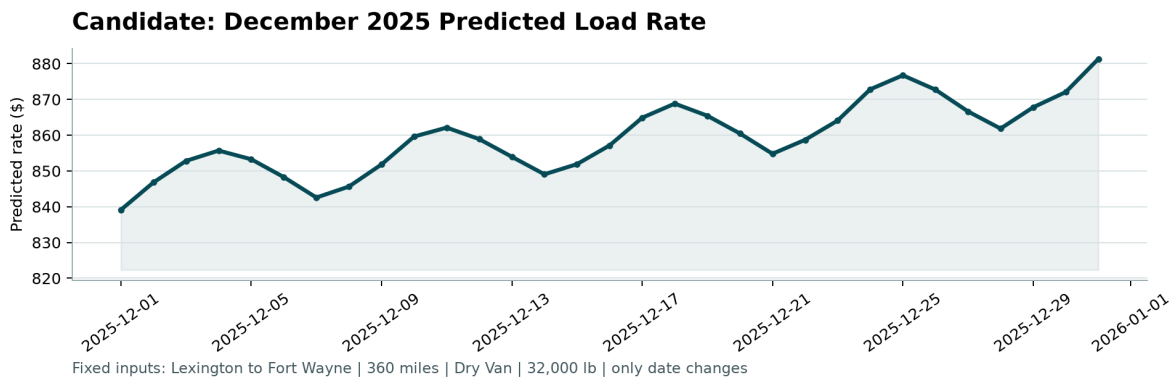


Figure 1: December 2025 chart produced by `score.py` from the submitted predictions.

2 Data and key findings

Distance and equipment. Distance explains most of the price: the correlation is 0.989 once the corrupted labels are removed (0.909 with them). Median rate per mile is \$2.05 for Dry Van, \$2.22 for Flatbed and \$2.31 for Reefer. Price grows less than proportionally with distance (log-log slope 0.87).

A daily market signal. `market_index` varies far more between days than within a day. Its weekly cycle bottoms on Sunday and Monday and peaks on Thursday. It averages 1.08 in training and 0.93 in validation, so it is a genuine input for November–December. Its price elasticity is 0.15.

Time structure that `market_index` does not explain. Once `market_index` is controlled for, prices still drift up about 6% from January to October. They also climb through every quarter-end month (March, June, September) and reset on the 1st. From the first day of the month to the last, Flatbed rises 8.1%, Reefer 5.1% and Dry Van 2.5% (Figure 2; these are fitted values from the final model). December is a quarter-end month.

Unseen cities. Eight validation cities never appear in training (1,447 rows, 12%). In training, city effects are small (-3.7% to $+2.7\%$), almost identical whether the city is the pickup or the delivery ($r = 0.99$), and they track latitude ($r = -0.86$, Figure 4). A smooth function of the coordinates therefore carries over to unseen cities. The coordinates are shifted from the real cities but are consistent for each city, so they are used as given.

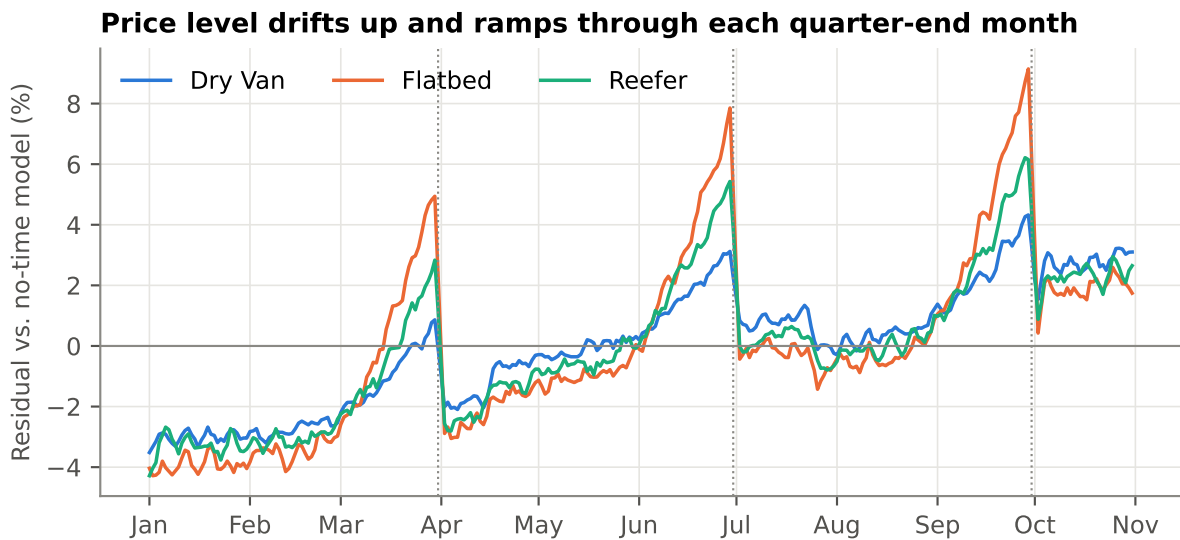


Figure 2: Daily mean residual by equipment from a model with no time terms (3-day mean). Dotted lines mark quarter ends.

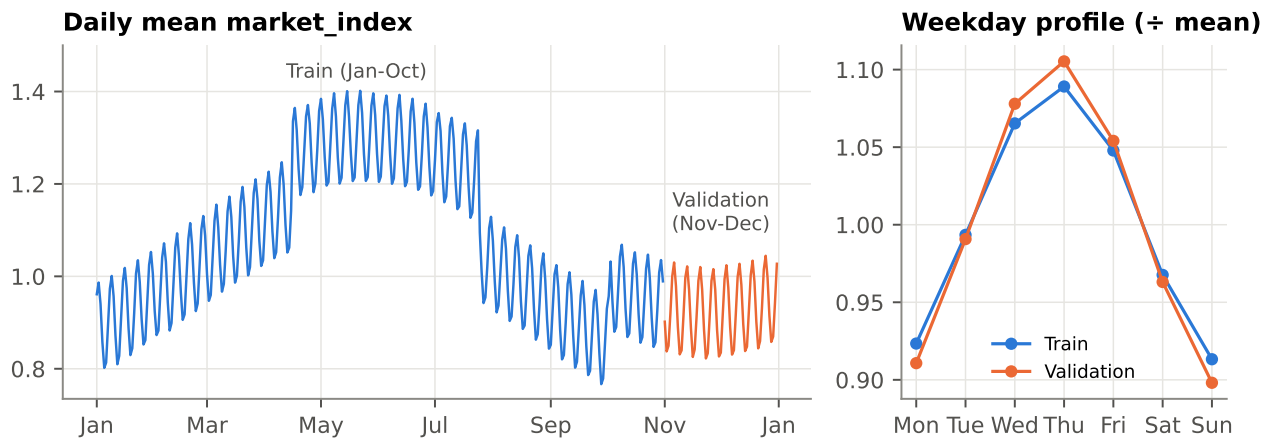


Figure 3: market_index: lower in November–December, with the same weekly cycle.

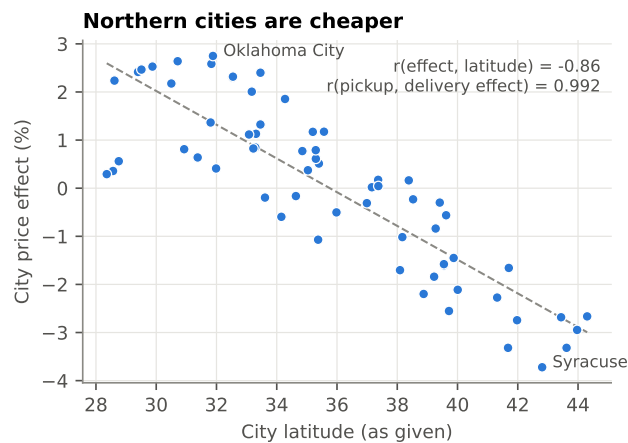


Figure 4: City price effect against latitude.

quote_signal is noise. Within lane $\times$ equipment groups, after weight, market_index and time terms, adding quote_signal leaves the residual SD unchanged (0.0226 without it, 0.0226 with it). Its apparent signal comes

from the equipment mix. Adding it to the hybrid worsens the fold-average MAE (\$86.7 vs \$85.2), and removing it improves Codex’s LightGBM (\$113.8 → \$102.9). It is dropped.

3 Data-quality issues and fixes

Issue	Train	Validation	Handling
Weight missing	300	165	Fill with training median $ weight $; w_miss flag
Weight negative (sign flip)	292	145	Absolute value; w_neg flag
Weight at cap $ w = 47,500$	1,204	301	Keep (censored); w_cap flag
of which -47,500	13	4	Absolute value
Weight at floor $ w = 5,000$	32	7	Keep; w_floor flag
market_index missing	374	249	Fill with that date’s mean
Distance at 70 mi floor	48	21	Keep (clipped, not an error)
Corrupted label ($ \log resid \geq 0.25$)	677	—	Dropped from training only (detector refitted inside each fold)
high cluster (median $\times 3.33$, range $\times 2.21$ -5.26)	340	—	
low cluster (median $\times 0.29$, range $\times 0.17$ -0.45)	337	—	

Table 1: Data-quality rules: counts on the training and validation files, and how each is handled.

Corrupted labels are found with a two-pass robust fit. The first pass fits the structural model on log price with least squares. The second refits it on rows with $|residual| < 0.30$. Rows with $|residual| \geq 0.25$ are then flagged. The clusters are far from the clean rows: no residual falls between 0.12 and 0.80 (Figure 5), so cut-offs of 0.20, 0.25 and 0.30 flag the same rows.

The detector is refitted inside every fold, and flagged rows are removed from training only; every validation row is predicted. The other repairs touch inputs only: missing values get flags, so the model can learn whatever their absence means.

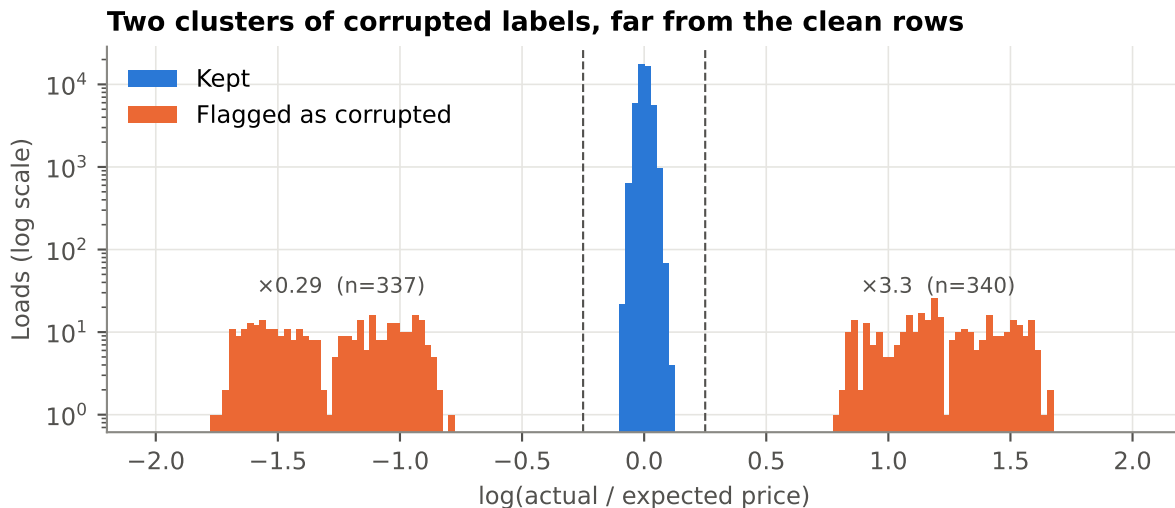


Figure 5: $\log(\text{actual}/\text{expected})$ on the full training set (log count scale).

4 Split and validation approach

The validation set is the two months after the training data, so every split looks forward in time. Three expanding folds test August, September and October, each trained on all earlier data. A fourth block, **h2**, trains through August and tests September–October: like November–December, it is one to two months past the end of training, and it spans a quarter end. Everything is refitted from scratch in each fold, including the label cleaning and the

weight imputation.

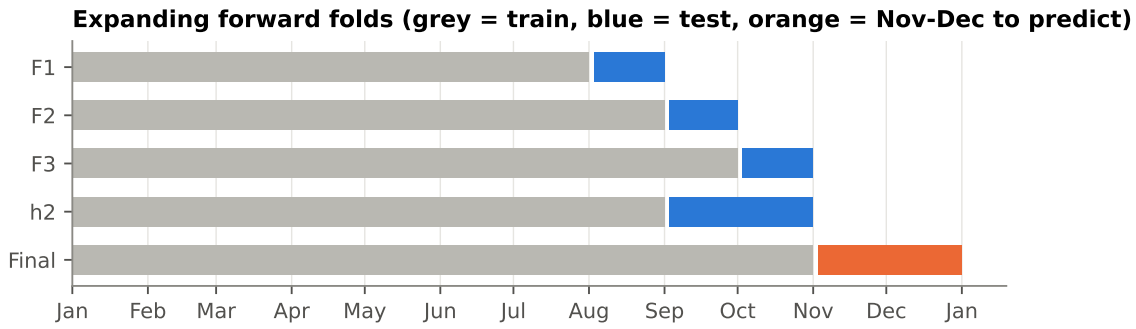


Figure 6: Folds. The final model trains on all of January–October.

Model selection used raw MAE averaged over F1–F3, with h2 MAE as the tie-break. Raw labels include the corruption, as Spotter’s labels presumably do, and MAE is robust to it. RMSE is not: every model lands between \$623 and \$640, because the corrupted rows dominate the squared error. Clean-label MAPE (corrupted rows removed, for reporting only) shows the accuracy on legitimate prices.

Split (MAE)	Hybrid	LightGBM-L1
Random 10% holdout (all months)	\$91.5	\$95.3
Forward: train Jan–Sep, test Oct	\$95.4	\$110.6
Unseen cities: 8 withheld, test Sep–Oct	\$88.2	\$110.6

Table 2: The same two models under three splits.

A random split makes the two models look about equal (\$91.5 vs \$95.3). Going forward in time, the tree model loses ground (\$95.4 vs \$110.6), so a random split would have picked the wrong model. The unseen-city simulation withholds 8 training cities from training and tests on the September–October loads that touch them. The hybrid holds up (\$88.2); the tree model, which relies on city identities, does not (\$110.6).

5 Model

Stage 1. Ordinary least squares on $\log(\text{posted_rate})$, fitted on the cleaned rows. The inputs are:

- log distance;
- log weight, with a hinge term for loads under 12,000 lb, and a missing-weight flag;
- log market_index and equipment dummies;
- a quadratic surface in pickup and delivery coordinates;
- a linear trend in months;
- a quarter-end ramp per equipment: 0 on the first day of March, June, September or December, rising to 1 on its last day, and 0 in other months.

Stage 2. LightGBM on the stage-1 residuals. Its inputs are distance, weight, market_index, equipment, coordinates, weekday, position within the month, and pickup and delivery city as categorical features. It has no trend input, so it cannot undo the trend. Rows that touch a city absent from training use a second stage-2 model without the city features. The prediction is $\exp(\text{stage 1} + \text{stage 2})$.

Why not boosted trees alone. A tree predicts a constant beyond the last date it saw, so it cannot carry the drift or a quarter-end ramp into the future. In the h2 backtest its error moves downward through the September ramp (Figure 7), and it cannot know that December will ramp. The linear stage carries time forward; the trees handle the non-linear parts that do not depend on the date.

Removing the trend or market_index roughly doubles the error (Table 3), so handling time is the main problem.

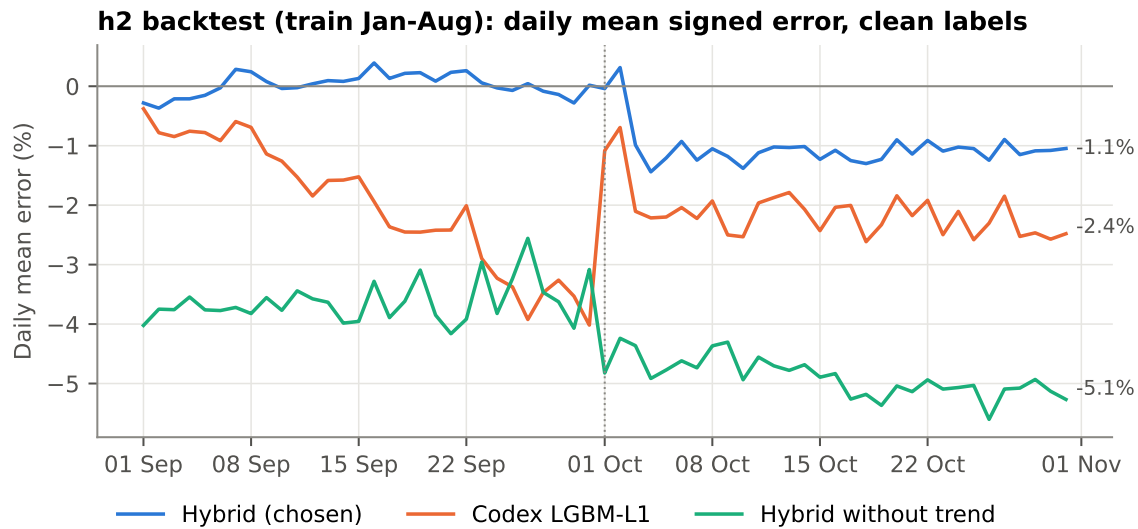


Figure 7: h2 backtest: daily mean signed error on clean labels (end labels show the last-5-day mean).

Model	Aug	Sep	Oct	h2	Fold avg	MAPE	Clean MAPE
Hybrid (chosen)	79.9	80.4	95.4	88.5	85.2	3.83	1.33
Hybrid, no city categoricals	81.4	81.9	96.6	89.9	86.6	3.89	1.39
Hybrid + quote_signal	85.4	79.9	94.9	87.9	86.7	3.90	1.40
Hybrid, untuned defaults	81.9	82.4	97.1	90.3	87.1	3.91	1.42
Hybrid 50/50 with Codex XGB	94.2	92.2	97.3	100.4	94.6	4.27	1.78
Hybrid without quarter-end ramp	94.4	107.3	90.5	102.5	97.4	4.33	1.83
Structural only (stage 1)	92.8	95.1	106.2	101.3	98.0	4.42	1.93
Codex LGBM-L1 without quote_signal	85.3	116.2	107.2	117.6	102.9	4.64	2.16
Codex blend 0.75 XGB / 0.25 LGBM	118.2	115.6	107.2	118.9	113.7	5.09	2.63
Codex LGBM-L1	119.5	111.4	110.6	114.8	113.8	5.08	2.62
Codex XGB-L1	121.7	119.5	109.4	122.3	116.9	5.26	2.80
Linear, city indicators, no time terms	110.1	174.9	133.0	165.7	139.3	5.95	3.56
Hybrid without trend	120.8	136.1	163.7	154.2	140.2	6.05	3.68
Hybrid without trend or ramp	109.0	174.6	160.8	173.2	148.1	6.38	4.01
Hybrid without market_index	197.3	205.8	101.1	169.2	168.1	7.41	4.85

Table 3: Forward-fold results. MAE in dollars on raw labels; MAPE and clean MAPE in percent, averaged over F1–F3. The Codex rows reproduce an earlier candidate solution: tree models on raw dollars with every feature, including quote_signal and calendar features.

Tuning. Before any tuning run, a small grid was committed to the repository, along with the selection rule. The search went one setting at a time. It chose a linear trend (over a step per quarter or a damped trend), 1,000 trees with 31 leaves, L2 loss, and city categoricals. That improved the fold-average MAE from \$87.1 to \$85.2.

One change was made after tuning. The folds contain no unseen cities, and in the unseen-city simulation the city categoricals made results worse (\$89.6 vs \$88.2 with routing). So rows that touch an unseen city use the model without them; this leaves every fold score unchanged. Results are stable across seeds (h2 MAE \$88.5 / \$88.5 / \$88.5).

6 December prediction

The chart inputs contain no market_index. Each December day gets the mean market_index of that day’s loads in validation.csv. This is an input feature Spotter supplied for the same dates, not a label, and it is how the model is applied to the validation loads too. Dropping it (with weekday dummies in its place) raises the h2 clean

MAPE from 1.37% to 4.76%.

The resulting chart reflects four things: the lower December market_index, the weekly cycle, the continuing drift, and the Dry Van quarter-end ramp. It averages \$859. A Codex-style LightGBM trained without market_index draws an almost flat line (\$835–\$847, Figure 8), because a tree has no way to extend time effects beyond October.

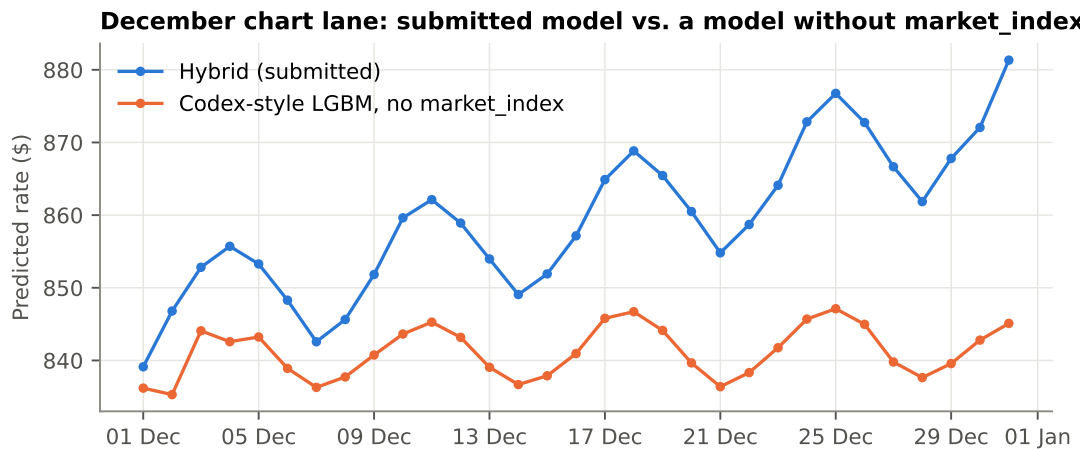


Figure 8: December chart lane: the submitted hybrid against a tree model without market_index.

Sanity checks. On the 12,000 validation predictions:

- no value is missing or non-positive;
- median rate per mile is \$2.07 for Dry Van (training \$2.05), \$2.26 for Flatbed (\$2.22) and \$2.35 for Reefer (\$2.31);
- rows touching an unseen city average \$2.23/mi, against \$2.22/mi for the rest.

7 Limitations and next steps

- **Extrapolation.** The linear trend and the December ramp are projected two months beyond the data. The ramp held in 3 of 3 quarters but cannot be checked for December. In the backtests the model under-predicted October, the month after a quarter end, by 1.0%.
- **Holidays.** Holiday effects in late December cannot be learned from January–October data.
- **Label corruption.** If the validation labels contain the same ~1.4% corruption, raw MAE and especially RMSE will carry an error floor that no model can remove.
- **Next steps.** Quantile models for prediction intervals; drift monitoring on market_index and the residual level; monthly refits as new labels arrive; checking the corruption mechanism with the data owner.

Appendix: how to reproduce

```
python -m pip install -r requirements.txt
python -m src.run_all          # full pipeline, about 6 minutes on 16 cores
python -m src.tune             # optional: rerun the pre-registered tuning grid
python score.py --predictions validation_predictions.csv \
                --december-predictions data/december_chart_inputs.csv
```

Code map: src/clean.py (input repairs, corrupted-label detector), src/models.py (HybridModel, baselines), src/validate.py (folds, metrics, contrasts), src/tune.py (pre-registered grid), src/train.py and src/predict.py (final fit, outputs), src/report.py (this report; its numbers are generated into reports/generated/). The README maps every file.

Python 3.12, pandas 2.3.3, numpy 2.5.3, scikit-learn 1.9.1, LightGBM 4.7.0; seeds fixed at 0.